Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №7
на тему

КРИПТОГРАФИЯ С ИСПОЛЬЗОВАНИЕМ ЭЛЛИПТИЧЕСКИХ КРИВЫХ

БГУИР КП 6-05-0612-02 015 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф.Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается асимметричная схема шифрования, являющаяся аналогом алгоритма Эль-Гамаля на основе эллиптических кривых. Алгоритм Эль-Гамаля был предложен Тахером Эль-Гамалем в 1985 году и в классическом виде работает в мультипликативной группе вычетов по модулю простого числа p. Получатель выбирает закрытый ключ x и публикует открытый ключ $y = g^x \bmod p$. Для зашифрования сообщения m отправитель выбирает случайное число k и передаёт пару

$$c_1 = g^k \bmod p, \quad c_2 = m \cdot y^k \bmod p.$$

Получатель, зная x, вычисляет $y^k = c_1^x$ и восстанавливает $m = c_2 \cdot (c_1^x)^{-1} \bmod p$. Стойкость схемы основана на сложности задачи дискретного логарифмирования: по известным g и y невозможно за приемлемое время найти x.

В том же 1985 году Н. Коблиц и В. Миллер независимо предложили использовать в криптографии группу точек эллиптической кривой над конечным полем. Любой алгоритм, построенный на задаче дискретного логарифмирования, можно перенести в эту группу: умножение элементов заменяется сложением точек, а возведение в степень – умножением точки на число. Для группы точек эллиптической кривой не известны субэкспоненциальные алгоритмы дискретного логарифмирования, поэтому при той же стойкости ключи получаются значительно короче: ключ длиной 256 бит на эллиптической кривой соответствует ключу RSA длиной около 3072 бит.

В эллиптическом аналоге алгоритма Эль-Гамаля сообщение представляется точкой кривой M, а шифртекстом является пара точек. Особенностью схемы является вероятностный характер шифрования: из-за случайного числа k один и тот же открытый текст при каждом зашифровании даёт разный шифртекст.

Целью лабораторной работы является разработка программного средства на языке C++, реализующего генерацию ключей, зашифрование и расшифрование текстовых файлов по схеме, аналогичной алгоритму Эль-Гамаля, на основе эллиптических кривых.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для шифрования файлов на эллиптических кривых. Все алгоритмы реализованы без использования сторонних библиотек. Программа поддерживает генерацию ключевой пары длиной 256 или 512 бит, зашифрование файла открытым ключом, расшифрование закрытым ключом и проверку реализации на контрольном примере.

Программа разделена на несколько модулей. Модуль bigint реализует арифметику больших целых чисел, модуль ec – операции с точками эллиптической кривой и наборы параметров, модуль random – генерацию случайных чисел, модуль elgamal – представление сообщения точками кривой, зашифрование и расшифрование, а главный модуль main.cpp отвечает за разбор аргументов командной строки и вывод результатов.

В модуле bigint большое число хранится как массив 32-битных слов. Реализованы сложение, вычитание, умножение, деление с остатком по алгоритму Кнута, а также операции по модулю: сложение, вычитание, умножение, возведение в степень и вычисление обратного элемента по малой теореме Ферма $a^{-1} = a^{p-2} \bmod p$.

Используется эллиптическая кривая в форме Вейерштрасса над простым полем $\mathrm{GF}(p)$:

$$y^2 = x^3 + ax + b \bmod p.$$

Точки кривой вместе с бесконечно удалённой точкой O образуют группу. Сумма точек (x_1, y_1) и (x_2, y_2) вычисляется по формулам

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1,$$

где $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$ при сложении различных точек и $\lambda = \frac{3x_1^2 + a}{2y_1}$ при удвоении. Противоположной к точке (x, y) является точка $(x, -y)$, поэтому вычитание точек сводится к сложению с противоположной точкой. Для ускорения вычислений точки представляются в якобиевых координатах (X, Y, Z), где $x = \frac{X}{Z^2}$, $y = \frac{Y}{Z^3}$, а произведение kP вычисляется методом «удвоения и сложения» по двоичной записи числа k.

В программе используются стандартизованные наборы параметров: для ключей длиной 256 бит – id-GostR3410-2001-CryptoPro-A-ParamSet с модулем $p = 2^{256} - 617$, для ключей длиной 512 бит – id-tc26-gost-3410-12-512-paramSetA с модулем $p = 2^{512} - 569$. Каждый набор задаёт коэффициенты a и b, простой порядок q подгруппы и базовую точку P порядка q.

Закрытым ключом получателя является случайное число d, $0 < d < q$, а открытым ключом – точка

$$Q = dP.$$

Для получения случайных чисел используется криптостойкий генератор операционной системы. Ключи сохраняются в текстовые файлы: открытый ключ содержит длину ключа и координаты точки Q, закрытый – длину ключа и число d.

Перед зашифрованием сообщение необходимо представить точками кривой. Для этого используется вероятностный метод Коблица. Файл разбивается на блоки длиной B байт: $B = 30$ для ключа 256 бит и $B = 62$ для ключа 512 бит. Блок рассматривается как целое число m, и для $j = 0, 1, \ldots, 255$ проверяется значение

$x = 256m + j.$

Для первого j, при котором правая часть уравнения кривой $f(x) = x^3 + ax + b$ является квадратичным вычетом по модулю p, находится координата y, и точка $M = (x, y)$ представляет блок. Для обоих наборов параметров выполняется $p \equiv 3 \bmod 4$, поэтому квадратный корень вычисляется по формуле

$y = f(x)^{\frac{p+1}{4}} \bmod p,$

после чего проверяется равенство $y^2 = f(x)$. Квадратичным вычетом является примерно половина чисел, поэтому вероятность того, что ни одно из 256 значений не подойдёт, составляет около 2^{-256}. Обратное преобразование выполняется отбрасыванием младшего байта: $m = \left\lfloor \frac{x}{256} \right\rfloor$.

Зашифрование каждого блока выполняется следующим образом:

1 Представить блок точкой кривой M.

2 Сгенерировать случайное число k, $0 < k < q$.

3 Вычислить точки $C_1 = kP$ и $C_2 = M + kQ$.

4 Шифртекстом блока является пара точек (C_1, C_2).

Для каждого блока выбирается новое число k. Если бы одно и то же k использовалось для двух блоков, то разность $C_2 - C_{2'} = M - M'$ не зависела бы от ключа, и знание одного открытого текста раскрывало бы другой.

Получатель расшифровывает блок с помощью закрытого ключа:

$M = C_2 - dC_1.$

Правильность расшифрования следует из равенства

$C_2 - dC_1 = M + kQ - dkP = M + kdP - dkP = M.$

Противник знает точки P, Q и $C_1 = kP$, но для вычисления $kQ = dkP$ ему необходимо найти d или k, то есть решить задачу дискретного логарифмирования на эллиптической кривой.

Зашифрованный файл является текстовым. В его начале записываются длина ключа, исходный размер файла и число блоков, а затем для каждого блока – четыре координаты точек C_1 и C_2 в шестнадцатеричном виде. Каждые 30 байт открытого текста превращаются в две точки, то есть в четыре числа по 256 бит, поэтому размер данных при шифровании заметно увеличивается. При расшифровании программа проверяет, что точки лежат на кривой, а восстановленное число m помещается в блок. При использовании чужого закрытого ключа получаются случайные точки, и программа сообщает об ошибке.

Для запуска программы предусмотрены следующие операции:

```
elgamal keygen [-b 256|512] public.key private.key
elgamal encrypt public.key input.txt secret.enc
elgamal decrypt private.key secret.enc output.txt
elgamal test
```

Первая команда создаёт ключевую пару, вторая выполняет зашифрование файла, третья – его расшифрование, а четвёртая – проверку реализации. В ка-

честве контрольного примера используется пример из учебника У. Столлингса «Криптография и защита сетей»: кривая $y^2 = x^3 - x + 188 \bmod 751$, базовая точка $G = (0, 376)$ порядка 727, открытый ключ получателя $P_B = (201, 5)$, сообщение $P_m = (562, 201)$ и число $k = 386$. По учебнику шифртекстом должна быть пара точек $C_1 = (676, 558)$ и $C_2 = (385, 328)$. Расшифрование проверяется закрытым ключом $d = 58$, для которого $P_B = 58G$. Кроме того, проверяются представление блока точкой и обратное преобразование, зашифрование и расшифрование для обоих наборов параметров, различие шифртекстов одного и того же сообщения и отказ при расшифровании чужим ключом. Результат проверки продемонстрирован на рисунке 1.1.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/M3И/Lab7$ ./build/elgamal test
Textbook C1 = kG        OK
Textbook C2 = Pm + kPb  OK
Textbook decrypt        OK
Block <-> point         OK
256 encrypt/decrypt     OK
512 encrypt/decrypt     OK
Random ciphertext       OK
Wrong key               OK
Все тесты пройдены
```

Рисунок 1.1 – Проверка реализации

Все вычисленные значения совпали с приведёнными в учебнике, что подтверждает правильность реализации. Генерация ключевой пары и содержимое файла открытого ключа продемонстрированы на рисунке 1.2.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/M3И/Lab7$ ./build/elgamal keygen examples/publi
c.key examples/private.key
Открытый ключ: examples/public.key
Закрытый ключ: examples/private.key

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/M3И/Lab7$ cat examples/public.key
bits: 256
Qx: dd784445a971f074145eb98256faeead7c17ace491112a2d97168eb725a07f64
Qy: 2aff2f9fa5268f4f81f16130d8fdc17c5dabae6437912b2df5883fef13f798c8
```

Рисунок 1.2 – Генерация ключевой пары

Далее тестовый файл input.txt был зашифрован открытым ключом. Начало зашифрованного файла продемонстрировано на рисунке 1.3: служебные данные и координаты точек C_1 и C_2 первых двух блоков.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ ./build/elgamal encrypt examples/publ
ic.key examples/input.txt examples/secret.enc
Зашифровано: examples/secret.enc

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ head -5 examples/secret.enc
bits: 256
size: 590
blocks: 20
5fb060dd80878e4a48f22b4ac341ea5e0ac0205417a0ef3ab2b7683e21edac67 fa2831f6d886fc08151ea9725ecba11dd2e0955
78e0dc3270cb65a6a4ff69098 aca73eb3b91f83b91cf47801dcf9c58143ea4910bc3ee377108e640a01f4e2ef 19b72993c9632
54564ee438fb2f28693a780298b44f8dfefb6897dfada54d641
2dcb97a66c064cd01719fdaf2c5f9ea88ff15f043ecf363507d34dc33c6461e4 a5944239a3aaa9f455800b67a05a877bed0230f
f8b661e23c15d9c1fb68977f3 124729096c394530b5344e9781c7aa85aba1be5a7c0356061e59a97bd196e400 8c4f6261e2067
0200516ea0a2c23161ea4c22084c4a682b0589f0a877c733de4
```

Рисунок 1.3 – Зашифрование файла

Расшифрование файла закрытым ключом продемонстрировано на рисунке 1.4. Текст в файле output.txt совпадает с текстом в исходном файле input.txt, что подтверждает утилита cmp.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ ./build/elgamal decrypt examples/priv
ate.key examples/secret.enc examples/output.txt
Расшифровано: examples/output.txt

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ cat examples/output.txt
Лабораторная работа №7. Криптография с использованием эллиптических кривых.

Аналог алгоритма Эль-Гамаля: открытый текст представляется точкой кривой M,
отправитель выбирает случайное k и передаёт пару точек C1 = kP, C2 = M + kQ.
Получатель восстанавливает M = C2 - d*C1 с помощью закрытого ключа d.

The quick brown fox jumps over the lazy dog. 0123456789 !@#$%^&*()

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ cmp examples/input.txt examples/outpu
t.txt && echo "Файлы совпадают"
Файлы совпадают
```

Рисунок 1.4 – Расшифрование файла

Попытка расшифровать файл закрытым ключом другой ключевой пары завершается ошибкой, что продемонстрировано на рисунке 1.5.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ ./build/elgamal keygen examples/other
_public.key examples/other_private.key
Открытый ключ: examples/other_public.key
Закрытый ключ: examples/other_private.key

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab7$ ./build/elgamal decrypt examples/othe
r_private.key examples/secret.enc examples/other.txt
Ошибка: неверный ключ или повреждённый шифртекст
```

Рисунок 1.5 – Расшифрование чужим ключом

Таким образом, в ходе работы была реализована схема шифрования, аналогичная алгоритму Эль-Гамаля, на эллиптических кривых с длиной ключа 256 и 512 бит с поддержкой работы с файлами.

ВЫВОД

В ходе выполнения лабораторной работы был изучен эллиптический аналог алгоритма Эль-Гамаля и разработано программное средство на языке C++, реализующее генерацию ключевой пары, зашифрование и расшифрование текстовых файлов.

В программе самостоятельно реализованы арифметика больших целых чисел, операции сложения, вычитания и умножения точек эллиптической кривой, представление блоков сообщения точками кривой методом Коблица с вычислением квадратного корня по модулю p, а также зашифрование $(C_1, C_2) = (kP, M + kQ)$ и расшифрование $M = C_2 - dC_1$. Корректность реализации подтверждена совпадением результатов с контрольным примером из учебника.

Проведённое тестирование показало, что после последовательного зашифрования и расшифрования исходный файл восстанавливается полностью, повторное зашифрование одного и того же текста даёт другой шифртекст, а попытка расшифрования чужим ключом завершается ошибкой. Подтверждён и недостаток схемы: шифртекст заметно больше открытого текста, так как каждый блок превращается в две точки кривой.

В результате выполнения лабораторной работы были получены практические навыки реализации асимметричных криптосистем на эллиптических кривых, работы с большими целыми числами и представления произвольных данных точками эллиптической кривой.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – elgamal.h

```
#pragma once

#include "bigint.h"
#include "ec.h"

#include <cstdint>
#include <string>
#include <vector>

namespace elgamal {

struct PrivateKey {
    int bits = 256;
    BigInt d;

    void save(const std::string& path) const;
    static PrivateKey load(const std::string& path);
};

struct PublicKey {
    int bits = 256;
    Point Q;

    void save(const std::string& path) const;
    static PublicKey load(const std::string& path);
};

struct CipherBlock {
    Point C1, C2;
};

size_t blockBytes(const Curve& c);
Point encodeBlock(const Curve& c, const std::vector<uint8_t>& block);
std::vector<uint8_t> decodeBlock(const Curve& c, const Point& M);

CipherBlock encryptPoint(const Curve& c, const Point& Q, const Point& M,
                         const BigInt& k);
Point decryptPoint(const Curve& c, const BigInt& d, const CipherBlock& cb);

void generateKeys(int bits, PrivateKey& priv, PublicKey& pub);
std::string encrypt(const PublicKey& key, const std::vector<uint8_t>& data);
std::vector<uint8_t> decrypt(const PrivateKey& key,
                             const std::string& cipher);

std::vector<uint8_t> readFile(const std::string& path);
void writeFile(const std::string& path, const std::vector<uint8_t>& data);

}
```

Листинг А.2 – elgamal.cpp

```
#include "elgamal.h"

#include "random.h"

#include <fstream>
```

```
#include <iterator>
#include <map>
#include <sstream>
#include <stdexcept>

namespace elgamal {

namespace {

using Fields = std::map<std::string, std::string>;

std::string readText(const std::string& path) {
    const auto data = readFile(path);
    return std::string(data.begin(), data.end());
}

void writeText(const std::string& path, const std::string& text) {
    writeFile(path, std::vector<uint8_t>(text.begin(), text.end()));
}

bool splitField(std::string line, std::string& key, std::string& value) {
    if (!line.empty() && line.back() == '\r') line.pop_back();
    const size_t colon = line.find(": ");
    if (colon == std::string::npos) return false;
    key = line.substr(0, colon);
    value = line.substr(colon + 2);
    return true;
}

Fields readFields(const std::string& path) {
    std::istringstream in(readText(path));
    Fields f;
    std::string line, key, value;
    while (std::getline(in, line)) {
        if (line.empty() || line == "\r") continue;
        if (!splitField(line, key, value))
            throw std::runtime_error(path + ": неверная строка");
        f[key] = value;
    }
    return f;
}

const std::string& field(const Fields& f, const std::string& key,
                         const std::string& path) {
    const auto it = f.find(key);
    if (it == f.end())
        throw std::runtime_error(path + ": нет поля \"" + key + "\"");
    return it->second;
}

[[noreturn]] void badCipher() {
    throw std::runtime_error("неверный ключ или повреждённый шифртекст");
}

}

void PrivateKey::save(const std::string& path) const {
    std::ostringstream out;
    out << "bits: " << bits << "\n";
    out << "d: " << d.toHex(bits / 4) << "\n";
    writeText(path, out.str());
}
```

```
PrivateKey PrivateKey::load(const std::string& path) {
    const Fields f = readFields(path);
    PrivateKey k;
    k.bits = std::stoi(field(f, "bits", path));
    k.d = BigInt::fromHex(field(f, "d", path));
    if (k.d.isZero() || !(k.d < curveFor(k.bits).q))
        throw std::runtime_error(path + ": недопустимый закрытый ключ");
    return k;
}

void PublicKey::save(const std::string& path) const {
    std::ostringstream out;
    out << "bits: " << bits << "\n";
    out << "Qx: " << Q.x.toHex(bits / 4) << "\n";
    out << "Qy: " << Q.y.toHex(bits / 4) << "\n";
    writeText(path, out.str());
}

PublicKey PublicKey::load(const std::string& path) {
    const Fields f = readFields(path);
    PublicKey k;
    k.bits = std::stoi(field(f, "bits", path));
    k.Q = {BigInt::fromHex(field(f, "Qx", path)),
           BigInt::fromHex(field(f, "Qy", path)), false};
    if (!curveFor(k.bits).contains(k.Q))
        throw std::runtime_error(path + ": точка Q не лежит на кривой");
    return k;
}

size_t blockBytes(const Curve& c) {
    const size_t bits = c.p.bitLength() - 1;
    return bits > 8 ? (bits - 8) / 8 : 0;
}

Point encodeBlock(const Curve& c, const std::vector<uint8_t>& block) {
    const BigInt m = BigInt::fromBytesLE(block);
    for (uint32_t j = 0; j < 256; ++j) {
        Point M;
        if (c.liftX(m * BigInt(256) + BigInt(j), M)) return M;
    }
    throw std::runtime_error("не удалось представить блок точкой кривой");
}

std::vector<uint8_t> decodeBlock(const Curve& c, const Point& M) {
    const size_t n = blockBytes(c);
    const BigInt m = M.x / BigInt(256);
    if (M.infinity || m.bitLength() > 8 * n) badCipher();
    return m.toBytesLE(n);
}

CipherBlock encryptPoint(const Curve& c, const Point& Q, const Point& M,
                         const BigInt& k) {
    return {c.multiply(k, c.P), c.add(M, c.multiply(k, Q))};
}

Point decryptPoint(const Curve& c, const BigInt& d, const CipherBlock& cb) {
    return c.subtract(cb.C2, c.multiply(d, cb.C1));
}

void generateKeys(int bits, PrivateKey& priv, PublicKey& pub) {
    const Curve& c = curveFor(bits);
```

```
    priv.bits = pub.bits = bits;
    priv.d = randomBelow(c.q);
    pub.Q = c.multiply(priv.d, c.P);
}

std::string encrypt(const PublicKey& key,
                    const std::vector<uint8_t>& data) {
    const Curve& c = curveFor(key.bits);
    const size_t n = blockBytes(c), w = size_t(key.bits) / 4;
    const size_t blocks = (data.size() + n - 1) / n;

    std::ostringstream out;
    out << "bits: " << key.bits << "\n";
    out << "size: " << data.size() << "\n";
    out << "blocks: " << blocks << "\n";
    for (size_t b = 0; b < blocks; ++b) {
        std::vector<uint8_t> block(n, 0);
        for (size_t i = 0; i < n && b * n + i < data.size(); ++i)
            block[i] = data[b * n + i];
        CipherBlock cb;
        do {
            cb = encryptPoint(c, key.Q, encodeBlock(c, block),
                              randomBelow(c.q));
        } while (cb.C2.infinity);
        out << cb.C1.x.toHex(w) << ' ' << cb.C1.y.toHex(w) << ' '
            << cb.C2.x.toHex(w) << ' ' << cb.C2.y.toHex(w) << "\n";
    }
    return out.str();
}

std::vector<uint8_t> decrypt(const PrivateKey& key,
                             const std::string& cipher) {
    const Curve& c = curveFor(key.bits);
    std::istringstream in(cipher);
    std::string line, k, v;
    size_t size = 0, blocks = 0;
    for (const char* name : {"bits", "size", "blocks"}) {
        if (!std::getline(in, line) || !splitField(line, k, v) || k != name)
            badCipher();
        if (k == "bits" && std::stoi(v) != key.bits)
            throw std::runtime_error(
                "длина ключа не совпадает с шифртекстом");
        if (k == "size") size = std::stoull(v);
        if (k == "blocks") blocks = std::stoull(v);
    }
    const size_t n = blockBytes(c);
    if (blocks != (size + n - 1) / n) badCipher();

    std::vector<uint8_t> out;
    for (size_t b = 0; b < blocks; ++b) {
        std::string x1, y1, x2, y2;
        if (!(in >> x1 >> y1 >> x2 >> y2)) badCipher();
        const CipherBlock cb{
            {BigInt::fromHex(x1), BigInt::fromHex(y1), false},
            {BigInt::fromHex(x2), BigInt::fromHex(y2), false}};
        if (!c.contains(cb.C1) || !c.contains(cb.C2)) badCipher();
        const auto part = decodeBlock(c, decryptPoint(c, key.d, cb));
        out.insert(out.end(), part.begin(), part.end());
    }
    out.resize(size);
    return out;
}
```